Supercomputing, Built from Lab Desktops: Design and Controlled Evaluation of a Four-Node Commodity Cluster

Divyansh Shukla, Abhinav Dhoke, Sneha Kumari, Vanshika Wadhwa, and Ankush Kumar
Cluster Kit project
github.com/TheDivyanshShukla/cluster-kit

Abstract—Small clusters assembled from ordinary desktop computers can give students and small laboratories access to parallel computing without dedicated high-performance hardware. This paper presents Cluster Kit, a lightweight master–worker framework that applies the principles of cluster supercomputers at classroom scale. Nodes find each other automatically by UDP broadcast, and work is distributed over persistent TCP connections with pull-based self-scheduling and two-deep chunk pipelining, so no host lists, remote shells or per-node configuration are needed. We evaluate it it on four identical Ubuntu workstations (12 hardware threads each) connected by a wired LAN. The workload is a SHA-256 partial-preimage search. Each configuration (every node alone, and 2, 3 and 4 nodes) was repeated seven times in an interleaved order, and comparisons use exact permutation tests and bootstrap confidence intervals. Four nodes sustained a median throughput of 87.38 MH/s, a speed-up of 3.86 (95% CI 3.81–3.89) over the mean single node and a parallel efficiency of 96.4% relative to the sum of the nodes' individual rates. Every added node increased throughput significantly ($p = 0.0003$, the minimum attainable with seven repeats). The Karp–Flatt experimentally determined serial fraction stayed between 0.4% and 1.3%. Work was shared evenly (23.9–25.7% per node) without static partitioning. We also show that an earlier informal comparison reporting a 17-fold gain was confounded, and we discuss threats to validity.

Index Terms—cluster computing, parallel efficiency, speed-up, self-scheduling, service discovery, UDP broadcast, MPI, SHA-256

I. Introduction

High-performance computing (HPC) has traditionally relied on purpose-built machines, but since the Beowulf project [4] it has been understood that commodity computers on a commodity network can form a useful parallel machine. University laboratories contain many such computers that sit idle for much of the day. A framework that turns them into a temporary cluster with minimal setup has teaching value, because it makes concepts such as speed-up, efficiency and load balancing measurable, and practical value for workloads that split into independent tasks.

Performance claims about such systems are often based on single runs, mismatched hardware or different software on each side of a comparison, which makes them hard to interpret [5]. An earlier informal measurement of the system studied here had exactly this problem (Section VI-C).

This paper makes three contributions:

1) It describes the design of *Cluster Kit*, a master–worker engine written in Rust that uses pull-based self-scheduling and pipelined chunk delivery over a minimal binary TCP protocol (Section III).
2) It reports a controlled scaling experiment on four identical nodes, with repeated, interleaved measurements, confidence intervals and hypothesis tests (Sections IV–V).
3) It analyses the results with the Karp–Flatt metric, identifies a performance anomaly on the head node, and corrects an earlier confounded comparison (Section VI).

II. Background and Related Work

A. Scaling laws

For p processors, the speed-up is $S_p = T_1/T_p$ and the parallel efficiency is $E_p = S_p/p$. Amdahl's law [1] bounds the speed-up of a fixed-size problem with serial fraction f by $S_p \le 1/(f + (1-f)/p)$. Gustafson [2] observed that in practice the problem size grows with the machine, so the *scaled* speed-up of a fixed-time workload is $S_p = p - f(p-1)$. Karp and Flatt [3] proposed estimating the serial fraction from measurements,

$$e_p = \frac{1/S_p - 1/p}{1 - 1/p}, \tag{1}$$

where a constant e_p indicates an inherently serial component and a growing e_p indicates overhead that increases with p.

B. Distribution frameworks

MPI [6] is the standard for tightly coupled HPC applications but requires an identical software environment and launcher on every node. MapReduce [7] and Dask [8] provide higher-level task models with richer scheduling and data movement at the cost of larger runtimes. Volunteer-computing systems such as BOINC [9] distribute independent work units to untrusted machines over the Internet. Cluster Kit sits at the opposite end: a single small binary for a trusted LAN, intended for workloads that split into independent ranges.

C. Self-scheduling

When tasks are assigned statically, the slowest node determines the completion time. Self-scheduling lets each worker request work when idle; chunk size trades scheduling overhead against load imbalance at the end of the job [10].

D. Workload

SHA-256 [11] maps any input to a 256-bit digest that is, for practical purposes, uniformly distributed. Finding an input whose digest begins with k zero hexadecimal digits therefore needs 16^k attempts on average and cannot be done faster than by trying inputs. This is the structure of proof-of-work in Bitcoin [12], and it makes a clean benchmark: tasks are independent, inputs and outputs are tiny, and the expected number of matches is known in advance.

III. System Design

Cluster Kit (Fig. 1) consists of a single executable, `ck`, that runs in one of three roles.

Head. The head accepts a job, defined as an integer range $[a, b)$, a chunk size and a task, and dispatches chunks on request. It records which chunks each worker holds, so that if a worker disconnects its unfinished chunks are requeued for others. It aggregates results and serves a live dashboard over HTTP. By default the head also starts a local worker, so a one-node cluster is a valid configuration.

Discovery (UDP). The head periodically broadcasts a small UDP beacon on port 7701. A worker started anywhere on the same subnet listens for the beacon, learns the head's address and connects; an explicit address can be given when broadcasts are blocked. UDP suits this role because a broadcast reaches every machine on the subnet in a single datagram and needs no prior knowledge of addresses, and a lost beacon is harmless since the next one follows shortly. Nodes can therefore join or leave a running cluster without editing any host list.

Transport (TCP). Chunks and results, by contrast, must not be lost or reordered, so each worker keeps one persistent TCP connection to the head on port 7700. Keeping the connection open avoids a handshake per chunk.

Workers. After connecting, a worker requests work. Each chunk is divided among all hardware threads using the Rust *rayon* data-parallel library [13]. The worker keeps a pipeline of two chunks: while one is being computed, the next has already arrived, which hides network latency. A returned RESULT also serves as the request for the next chunk, so faster nodes automatically receive more work (self-scheduling).

Protocol. Messages use a fixed frame of one byte for the message type, a 32-bit little-endian length and a binary payload. A RESULT carries the job and range identifiers, the result body and the compute time the worker spent on the chunk, which the head uses for the per-node statistics in Section V-C.

TABLE I
Experimental testbed

Nodes	4 identical desktops (mainboard MS-7D22)
Per node	12 hardware threads, 7.1 GiB RAM
OS	Ubuntu 26.04.1 LTS, Linux 7.0
Network	wired Ethernet LAN, RTT < 1 ms
Roles	N1 (192.168.10.122): head + worker
	N2 (.200), N3 (.3), N4 (.5): workers
Engine	`ck` (Rust), CPU only

Tasks. Besides the built-in SHA-256 search used here, a job can run an arbitrary program on each sub-range, or the head can send a script to workers that do not have it. The repository also contains a Python/Dask engine and a distributed video-analytics system; neither is evaluated in this paper.

IV. Methodology

A. Testbed

Table I summarises the hardware and Fig. 2 shows the physical set-up. All four nodes are the same model and ran the same operating system and the same `ck` binary, built once on N1 and copied to the other nodes.

B. Workload and configurations

Each run hashes the integers $1, \dots, n$ with SHA-256 and counts digests that start with `000000` (probability 16^{-6} per hash). Seven configurations were measured: each node alone (N1–N4), and clusters of $p = 2, 3, 4$ nodes, built by adding N2, N3 and N4 to N1 in that order. The range size n was chosen per configuration so that a run lasts about 3 s (Table II); the work per node is therefore approximately constant, which is the fixed-time (scaled) regime of Gustafson [2].

C. Protocol

After one warm-up run, the seven configurations were run seven times each in round-robin order, so that slow drifts such as heat or background activity affect all configurations similarly instead of being confounded with one of them [5]. Throughput is $X = n/t$, where t is the wall-clock time from job submission to the last result, as measured by the client.

D. Metrics

Because N1 turned out to be slower than the other nodes (Section VI-B), a speed-up relative to N1 alone would overstate the benefit of clustering. We therefore report three measures:

$$S_p = X_p / \bar{X}_1, \qquad \bar{X}_1 = \tfrac{1}{4} \textstyle\sum_{i=1}^{4} X_1^{(i)}, \tag{2}$$

$$E_p = X_p \Big/ \textstyle\sum_{i \in \mathcal{N}_p} X_1^{(i)}, \tag{3}$$

where $X_1^{(i)}$ is the median rate of node i alone and $\mathcal{N}_p$ is the set of nodes used. Equation (3) accounts for heterogeneous node speeds: $E_p = 1$ means the cluster is exactly as fast as

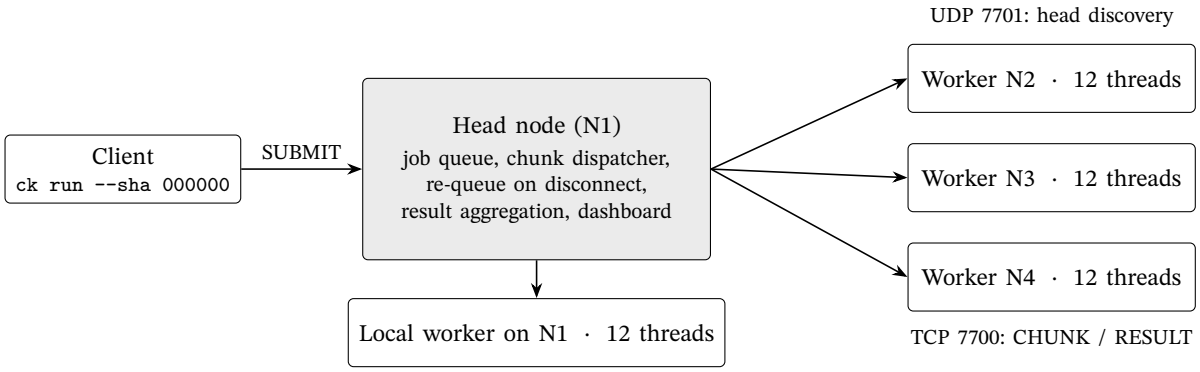


Fig. 1. Cluster Kit architecture. Workers discover the head by UDP broadcast and keep one persistent TCP connection. Each worker holds up to two chunks (one computing, one queued); every RESULT message also requests the next chunk. The head node additionally runs a local worker.

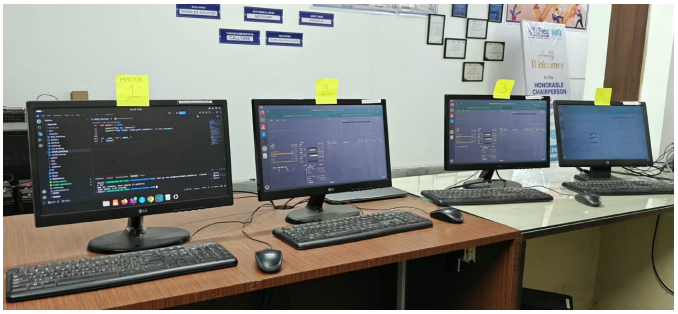


Fig. 2. The testbed: four identical lab desktops on one wired LAN. The node labelled "Master 1" (N1) is the head; nodes 2–4 are workers.

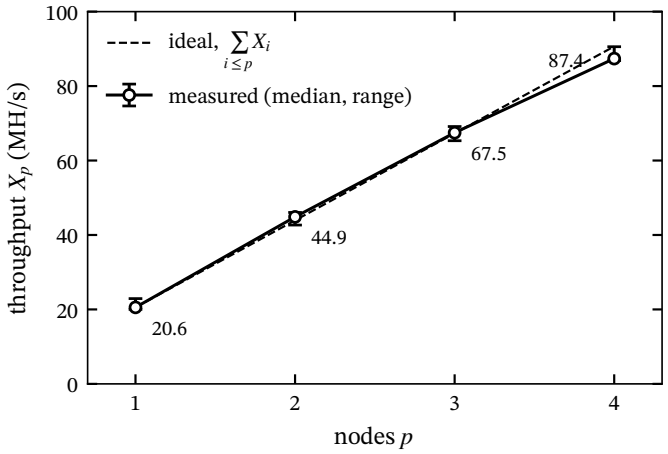


Fig. 3. Median throughput versus number of nodes; bars show the range of the seven repeats. The dashed line is the sum of the participating nodes' individual rates.

its members working separately. For comparison we also give the speed-up relative to N1. The serial fraction e_p is computed from S_p with (1).

E. Statistics

Medians are used as point estimates. Confidence intervals for means use the t distribution. Confidence intervals for speed-ups are percentile bootstrap intervals with 5,000 resamples [14]. Differences between configurations are tested with one-sided exact permutation tests on the seven repeat throughputs of each [15]. With $7 + 7$ observations there are $\binom{14}{7} = 3{,}432$ splits, so the smallest attainable p-value is $1/3{,}432 \approx 0.0003$.

V. Results

A. Throughput scaling

Table II lists all configurations and Fig. 3 shows throughput against the number of nodes. Throughput rose from 20.55 MH/s on N1 alone to 87.38 MH/s on four nodes, closely following the ideal line given by the sum of the participating nodes' individual rates.

B. Speed-up, efficiency and serial fraction

Table III and Fig. 4 give the derived metrics. Relative to the mean single node, four nodes are 3.86 times faster (95% CI 3.81–3.89), an efficiency of 96.4%. For $p = 2$ and 3 the efficiency is slightly above 100%; the excess (0.3–2.1%) is within the run-to-run variation of the single-node baselines and is not evidence of super-linear behaviour. The Karp–Flatt serial fraction stays between 0.004 and 0.013 and does not grow steadily with p, so there is no sign of overhead increasing quickly with cluster size over this range.

C. Per-node behaviour and load balance

Fig. 5 and Table IV compare each node's rate alone with its rate while computing inside the four-node run, using the per-chunk compute times reported by the workers. Nodes N2–N4 kept their stand-alone rates to within 1%. N1 computed 3.7% faster inside the cluster than in its stand-alone runs, which supports the view that those stand-alone runs were depressed by background load (Section VI-B). The the numbers were divided almost evenly (23.9–25.7%) although no node was given a fixed share. N1 received the smallest share, consistent with its lower speed. "Busy" is the sum of a node's chunk compute times divided by the job's elapsed time; it was 95.7–98.5%, so less than 5% of the time was spent waiting for work.

TABLE II
THROUGHPUT OF ALL CONFIGURATIONS (SEVEN REPEATS EACH). MH/S = 10^6 HASHES PER SECOND. CV = COEFFICIENT OF VARIATION.

Configuration	Nodes	Threads	n per run	Median (MH/s)	Mean ± 95% CI (MH/s)	CV (%)	Matches
N1 (head)	1	12	68,000,000	20.55	20.92 ± 0.93	4.78	5
N2	1	12	70,000,000	23.38	23.32 ± 0.17	0.78	5
N3	1	12	70,000,000	23.37	23.35 ± 0.14	0.67	5
N4	1	12	70,000,000	23.35	23.34 ± 0.04	0.20	5
$p = 2$ (N1–N2)	2	24	130,000,000	44.86	44.69 ± 1.16	2.80	8
$p = 3$ (N1–N3)	3	36	200,000,000	67.48	67.20 ± 1.50	2.40	16
$p = 4$ (N1–N4)	4	48	270,000,000	87.38	87.85 ± 1.15	1.42	21

TABLE III
SCALING METRICS (MEDIANS). S_p IS RELATIVE TO THE MEAN SINGLE NODE $\bar{X}_1 = 22.66$ MH/S.

p	X_p	S_p [95% CI]	vs N1	E_p (%)	e_p
1	20.55	0.91 [0.89, 0.95]	1.00	100.0	–
2	44.86	1.98 [1.92, 2.03]	2.18	102.1	0.010
3	67.48	2.98 [2.87, 3.05]	3.28	100.3	0.004
4	87.38	3.86 [3.81, 3.89]	4.25	96.4	0.013

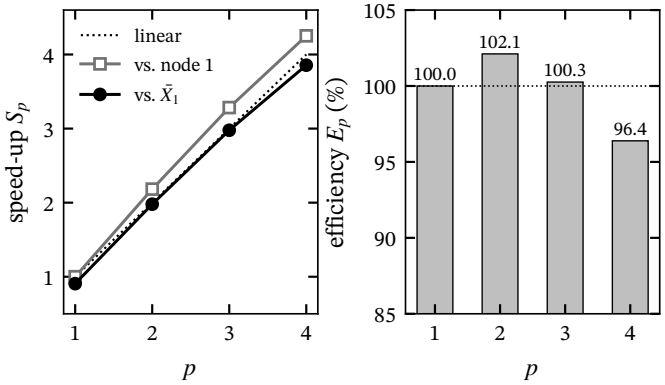


Fig. 4. Left: speed-up relative to node N1 and to the mean single node $\bar{X}_1$. Right: efficiency E_p as defined in (3).

TABLE IV
PER-NODE BEHAVIOUR IN THE FOUR-NODE RUN

Node	Alone (MH/s)	In cluster (MH/s)	Share (%)	Busy (%)
N1	20.55	21.31	23.9	98.5
N2	23.38	23.23	25.7	97.1
N3	23.37	23.12	25.2	95.8
N4	23.35	23.15	25.2	95.7

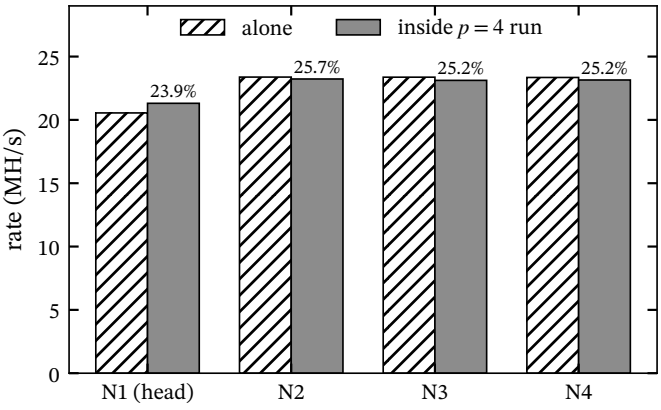


Fig. 5. Each node's hashing rate alone and while part of the four-node run. Labels give the node's share of the numbers hashed.

D. Variability and correctness

Fig. 6 shows the spread of the repeats. Nodes N2–N4 alone varied by less than 1% (CV), the clusters by 1.4–2.8%, and N1 alone by 4.8%. Every repeat of a configuration returned the same number of matching digests (Table II). The expected count is $n/16^6$, i.e. 4.1, 7.7, 11.9 and 16.1 for 70, 130, 200 and 270 million numbers; the observed counts are consistent with these under Poisson variation. This confirms that the distributed runs covered the same input ranges as intended, although it is a consistency check rather than verification against an independent implementation.

E. Hypothesis tests

Table V shows that each added node increased throughput, and that four nodes outperform the fastest single node. All four tests reached the smallest attainable p-value, meaning every repeat of the larger configuration was faster than every repeat of the smaller one. The results remain significant at $\alpha = 0.01$ after Bonferroni correction for four tests.

F. Live operation

Fig. 7 shows the cluster during a run of the project's original example program, `simple_sha256.py`, submitted from N1 with `kit py run examples/simple_sha256.py --cluster --n 5e7`. The program reports that it ran on the cluster through the Rust engine with four workers and finished in 3.87 s. In this mode each chunk is processed by a Python script, so per-node rates (3.4–4.5 M numbers/s) are much lower than those of the built-in SHA-256 routine used in Table II. The head's dashboard lists all four nodes at 95–99% CPU utilisation, and its task stream shows chunks finishing on every node at the same time. These photographs illustrate operation only; all quantitative results above come from the controlled benchmark.

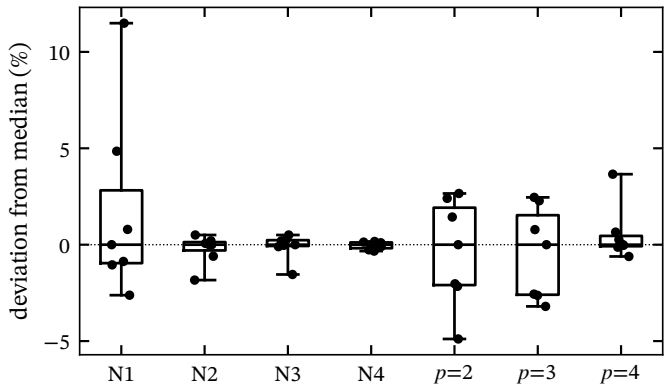


Fig. 6. Run-to-run variability: deviation of each repeat from its configuration's median (box: quartiles, whiskers: range, dots: individual repeats).

TABLE V
ONE-SIDED EXACT PERMUTATION TESTS (7 VS. 7 REPEATS)

Alternative hypothesis	Gain	p
$X_2 > X_1^{(1)}$ (N1 alone)	2.18×	0.0003
$X_3 > X_2$	1.50×	0.0003
$X_4 > X_3$	1.29×	0.0003
$X_4 > X_1^{(2)}$ (fastest single node)	3.74×	0.0003

VI. Discussion

A. Why the scaling is near-linear

The workload has no communication between tasks, and each chunk's input (two integers) and output (a count) are tiny compared with its compute time. Pull-based scheduling adapts to node speed without configuration, and the two-chunk pipeline keeps workers supplied while results travel back. The remaining losses (about 3.6% at $p = 4$) are consistent with job start-up and the uneven finish of the last chunks, which have a larger relative cost in a run of only about 3 s; longer jobs should therefore scale at least as well.

B. The head-node anomaly

Node N1 was 12.1% slower alone than the mean of the identical N2–N4 (20.55 vs. 23.37 MH/s) and was the least stable configuration. The most plausible explanation is background load: N1 was the head and was also hosting the project's video-analytics service during the session. This was not isolated experimentally. Because of it, speed-ups relative to N1 (4.25 at $p = 4$) overstate the benefit of clustering, and we treat S_p relative to $\bar{X}_1$ and the heterogeneity-aware E_p as the primary results.

C. Correction of an earlier informal comparison

An earlier project report compared a single Windows PC (36.98 s) with a four-worker cluster (2.165 s) on a run that found 766 matches, and reported a ratio of 17.08. That ratio is not a valid speed-up. The two sides used different hardware and operating systems and likely different engines (the Python/Dask path is much slower per hash than the Rust engine), and each figure came from a single run. For an embarrassingly parallel workload on four comparable nodes, the scaled speed-up cannot exceed roughly four, which agrees with the 3.86 measured here under controlled conditions.

TABLE VI
DESIGN COMPARISON WITH TYPICAL MPI DEPLOYMENTS

Aspect	Typical MPI	Cluster Kit
Finding nodes	host file written by hand	UDP broadcast, automatic
Launching	`mpirun` over SSH, keys on every node	one command per node
Work division	usually static by rank	dynamic, pulled on demand
Uneven nodes	slowest rank sets pace	faster nodes take more chunks
Node failure	job aborts by default	lost chunks re-queued
Communication	any pattern, collectives	master–worker only
Mixed OS	same MPI on all nodes	one portable binary

D. Comparison with MPI

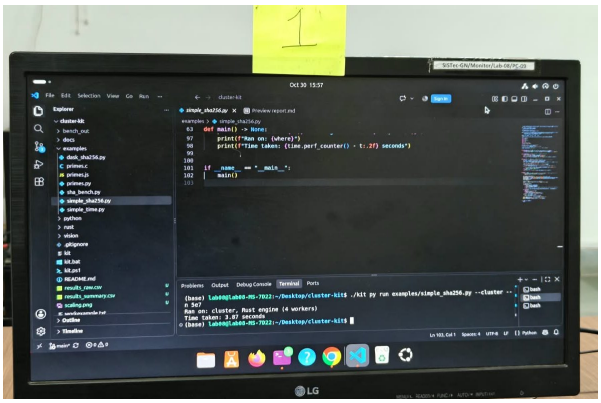
MPI [6] is the dominant programming model on production supercomputers, so it is the natural reference point. Table VI contrasts the two designs. The comparison concerns deployment and robustness, not speed: we did not run an MPI implementation of the workload, so we make no claim that Cluster Kit is faster. For a compute-bound workload such as this one, little room for improvement remains, since Cluster Kit already reaches 96.4% of the nodes' combined stand-alone throughput; a well-written MPI program would be expected to perform similarly. Cluster Kit's advantages lie elsewhere. Zero-configuration discovery removes host lists and remote-shell set-up. Pull-based scheduling absorbs differences in node speed, which matters on a heterogeneous lab floor. Re-queuing lets a job survive the loss of a node, whereas the default MPI error handler aborts the whole job. These properties trade some generality for ease of use: MPI supports arbitrary communication patterns between processes, while Cluster Kit is limited to independent tasks. A measured MPI baseline on the same testbed is planned as future work.

E. Practical implications

At the measured rates, hashing 10^{11} numbers would take 81 min on N1 and 19 min on four nodes. For the search problem itself, each additional zero digit multiplies the expected work by 16 (Table VII). A four-fold speed-up extends the feasible difficulty by only $\log_{16} 4 = 0.5$ digits, which illustrates why production proof-of-work systems use specialised hardware rather than general-purpose clusters.

F. Threats to validity

Internal validity. The background load on N1 was not controlled; the interleaved run order reduces but does not remove the effect of slow drift. Runs were short (about 3 s), so start-up costs are a larger fraction than in long



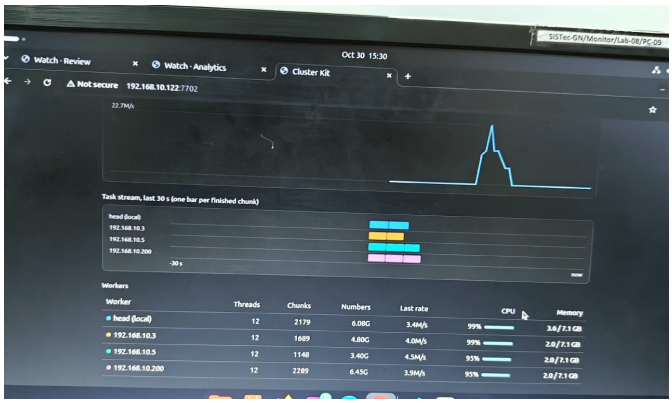


Fig. 7. Live operation on the testbed. Left: the head node after running `simple_sha256.py` on the cluster ("Ran on: cluster, Rust engine (4 workers)"). Right: the head's live dashboard (port 7702), with the per-chunk task stream and all four workers at 95–99% CPU utilisation.

TABLE VII
EXPECTED TIME TO FIND ONE DIGEST WITH k LEADING ZERO HEX DIGITS (16^k HASHES)

k	Expected hashes	N1 alone	4 nodes
6	1.68×10^7	0.82 s	0.19 s
8	4.29×10^9	3.5 min	49 s
10	1.10×10^{12}	14.9 h	3.5 h
12	2.81×10^{14}	158.5 d	37.3 d

jobs. *Construct validity.* Throughput was timed by the client and includes job submission and result collection, which is the user-visible time but mixes computation and coordination. Per-node "busy" values are derived from worker-reported compute times, not from operating-system counters. *External validity.* SHA-256 search is close to the best case for distributed computing. Workloads with large inputs, shared state or frequent communication will scale less well, and results on Wi-Fi or a congested network may differ. All measurements come from one session on one LAN.

VII. Conclusion and Future Work

On four identical commodity workstations, Cluster Kit achieved a speed-up of 3.86 over the mean single node at 96.4% parallel efficiency on a SHA-256 search workload, with every added node giving a statistically significant improvement, an even work split without manual tuning, and identical results across repeats. Simple design choices (self-scheduling, pipelined chunks and a compact binary protocol) were enough to reach near-linear scaling for independent tasks. Future work includes strong-scaling experiments with a fixed problem size and longer runs, isolating the head-node overhead by running the head on a dedicated machine, evaluating workloads with substantial data movement, measuring GPU-equipped workers, and scaling beyond four nodes to locate the point at which the single head becomes a bottleneck.

References

[1] G. M. Amdahl, "Validity of the single processor approach to achieving large scale computing capabilities," in *Proc. AFIPS Spring Joint Computer Conf.*, 1967, pp. 483–485.

[2] J. L. Gustafson, "Reevaluating Amdahl's law," *Commun. ACM*, vol. 31, no. 5, pp. 532–533, 1988.

[3] A. H. Karp and H. P. Flatt, "Measuring parallel processor performance," *Commun. ACM*, vol. 33, no. 5, pp. 539–543, 1990.

[4] T. Sterling, D. J. Becker, D. Savarese, J. E. Dorband, U. A. Ranawake, and C. V. Packer, "Beowulf: A parallel workstation for scientific computation," in *Proc. Int. Conf. Parallel Processing (ICPP)*, 1995.

[5] T. Hoefler and R. Belli, "Scientific benchmarking of parallel computing systems: Twelve ways to tell the masses when reporting performance results," in *Proc. Int. Conf. High Performance Computing, Networking, Storage and Analysis (SC15)*, 2015.

[6] Message Passing Interface Forum, *MPI: A Message-Passing Interface Standard, Version 4.0*, 2021.

[7] J. Dean and S. Ghemawat, "MapReduce: Simplified data processing on large clusters," in *Proc. 6th USENIX Symp. Operating Systems Design and Implementation (OSDI)*, 2004.

[8] M. Rocklin, "Dask: Parallel computation with blocked algorithms and task scheduling," in *Proc. 14th Python in Science Conf. (SciPy)*, 2015, pp. 130–136.

[9] D. P. Anderson, "BOINC: A system for public-resource computing and storage," in *Proc. 5th IEEE/ACM Int. Workshop on Grid Computing*, 2004, pp. 4–10.

[10] S. F. Hummel, E. Schonberg, and L. E. Flynn, "Factoring: A method for scheduling parallel loops," *Commun. ACM*, vol. 35, no. 8, pp. 90–101, 1992.

[11] National Institute of Standards and Technology, "Secure Hash Standard (SHS)," FIPS PUB 180-4, 2015.

[12] S. Nakamoto, "Bitcoin: A peer-to-peer electronic cash system," 2008. [Online]. Available: https://bitcoin.org/bitcoin.pdf

[13] Rayon contributors, "Rayon: A data parallelism library for Rust." [Online]. Available: https://github.com/rayon-rs/rayon

[14] B. Efron and R. J. Tibshirani, *An Introduction to the Bootstrap*. New York: Chapman & Hall, 1993.

[15] P. Good, *Permutation, Parametric, and Bootstrap Tests of Hypotheses*, 3rd ed. New York: Springer, 2005.

Appendix
Raw Measurements

Table VIII lists all 49 timed runs (benchmark identifier `20261030-150048-cpu`) in the order they were executed.

TABLE VIII
ALL TIMED RUNS

Config.	Rep.	n	Time (s)	MH/s	Matches
N1	1	68,000,000	2.968	22.91	5
N2	1	70,000,000	2.994	23.38	5
N3	1	70,000,000	3.042	23.01	5
N4	1	70,000,000	2.994	23.38	5
p=2	1	130,000,000	2.898	44.86	8
p=3	1	200,000,000	3.042	65.75	16
p=4	1	270,000,000	3.090	87.38	21
N1	2	68,000,000	3.156	21.55	5
N2	2	70,000,000	2.992	23.40	5
N3	2	70,000,000	2.989	23.42	5
N4	2	70,000,000	2.993	23.39	5
p=2	2	130,000,000	2.962	43.89	8
p=3	2	200,000,000	3.044	65.70	16
p=4	2	270,000,000	3.094	87.27	21
N1	3	68,000,000	3.344	20.33	5
N2	3	70,000,000	3.012	23.24	5
N3	3	70,000,000	2.995	23.37	5
N4	3	70,000,000	3.008	23.27	5
p=2	3	130,000,000	3.047	42.66	8
p=3	3	200,000,000	3.062	65.32	16
p=4	3	270,000,000	3.082	87.61	21
N1	4	68,000,000	3.309	20.55	5
N2	4	70,000,000	2.988	23.43	5
N3	4	70,000,000	2.980	23.49	5
N4	4	70,000,000	2.998	23.35	5
p=2	4	130,000,000	2.857	45.50	8
p=3	4	200,000,000	2.964	67.48	16
p=4	4	270,000,000	3.109	86.84	21
N1	5	68,000,000	3.283	20.71	5
N2	5	70,000,000	3.050	22.95	5
N3	5	70,000,000	2.996	23.36	5
N4	5	70,000,000	3.006	23.29	5
p=2	5	130,000,000	2.830	45.94	8
p=3	5	200,000,000	2.941	68.00	16
p=4	5	270,000,000	3.070	87.95	21
N1	6	68,000,000	3.398	20.01	5
N2	6	70,000,000	2.994	23.38	5
N3	6	70,000,000	2.987	23.43	5
N4	6	70,000,000	3.001	23.33	5
p=2	6	130,000,000	2.823	46.05	8
p=3	6	200,000,000	2.898	69.01	16
p=4	6	270,000,000	3.091	87.35	21
N1	7	68,000,000	3.338	20.37	5
N2	7	70,000,000	2.979	23.50	5
N3	7	70,000,000	2.998	23.35	5
N4	7	70,000,000	2.995	23.37	5
p=2	7	130,000,000	2.958	43.95	8
p=3	7	200,000,000	2.893	69.13	16
p=4	7	270,000,000	2.981	90.57	21